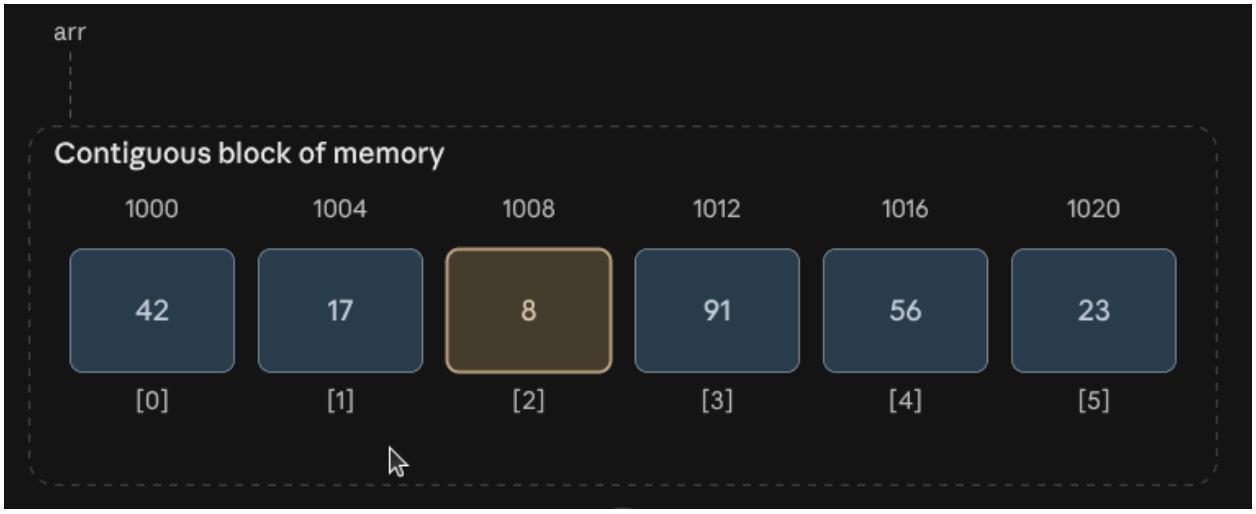
Arrays



That diagram is the core idea: elements sit back-to-back in memory, so the computer can jump straight to any element using simple arithmetic — no searching required.

Why indexing is O(1)

Because elements are the same size and packed contiguously, the address of any element is just `base_address + index × element_size`. That's one multiplication and one addition — constant time regardless of whether the array has 10 elements or 10 million. This is the single biggest advantage arrays have over most other structures.

Static vs. dynamic arrays

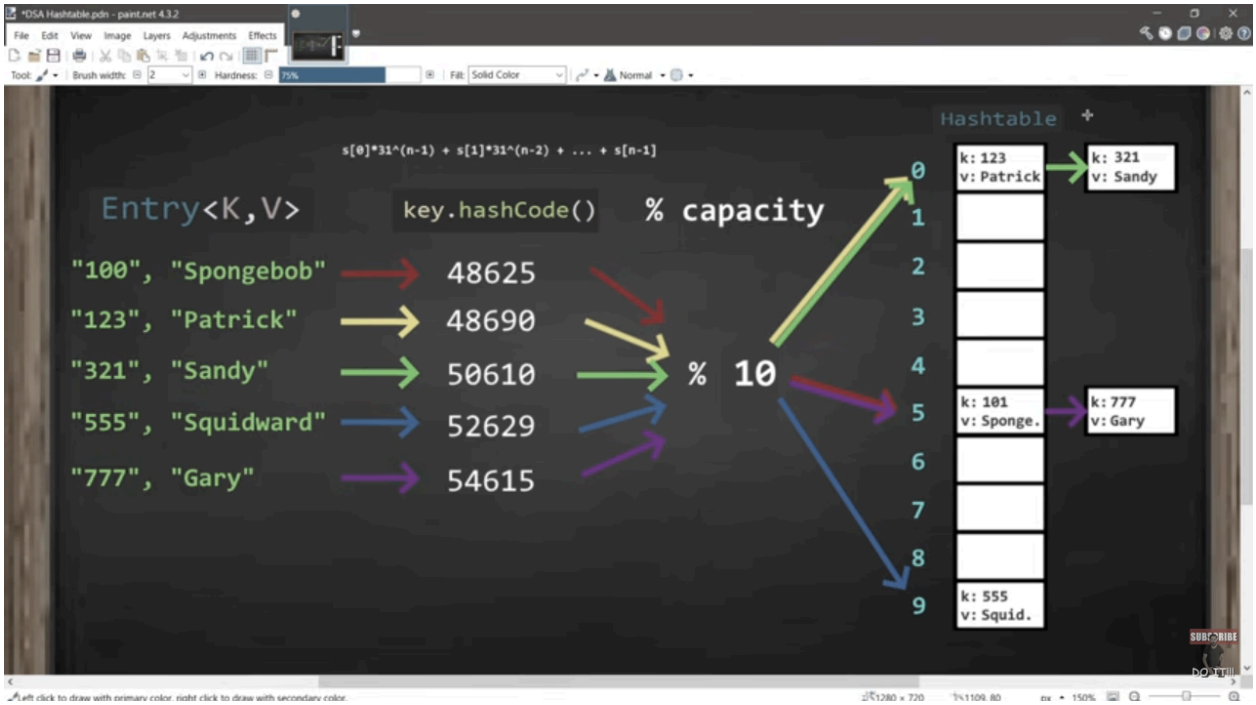
- A **static array** has a fixed size set at creation. Once it's full, you can't add more elements — you'd need to allocate a new, bigger array and copy everything over. C's `int arr[10]` is a classic static array.
- A **dynamic array** (Python's `list`, Java's `ArrayList`, C++'s `vector`, JavaScript's `Array`) looks resizable, but under the hood it's still a static array — the runtime just automatically allocates a new, larger block (often double the size) and copies elements over when it runs out of room. Because doubling happens rarely, the *average* cost of appending works out to O(1) — this is called **amortized constant time**.

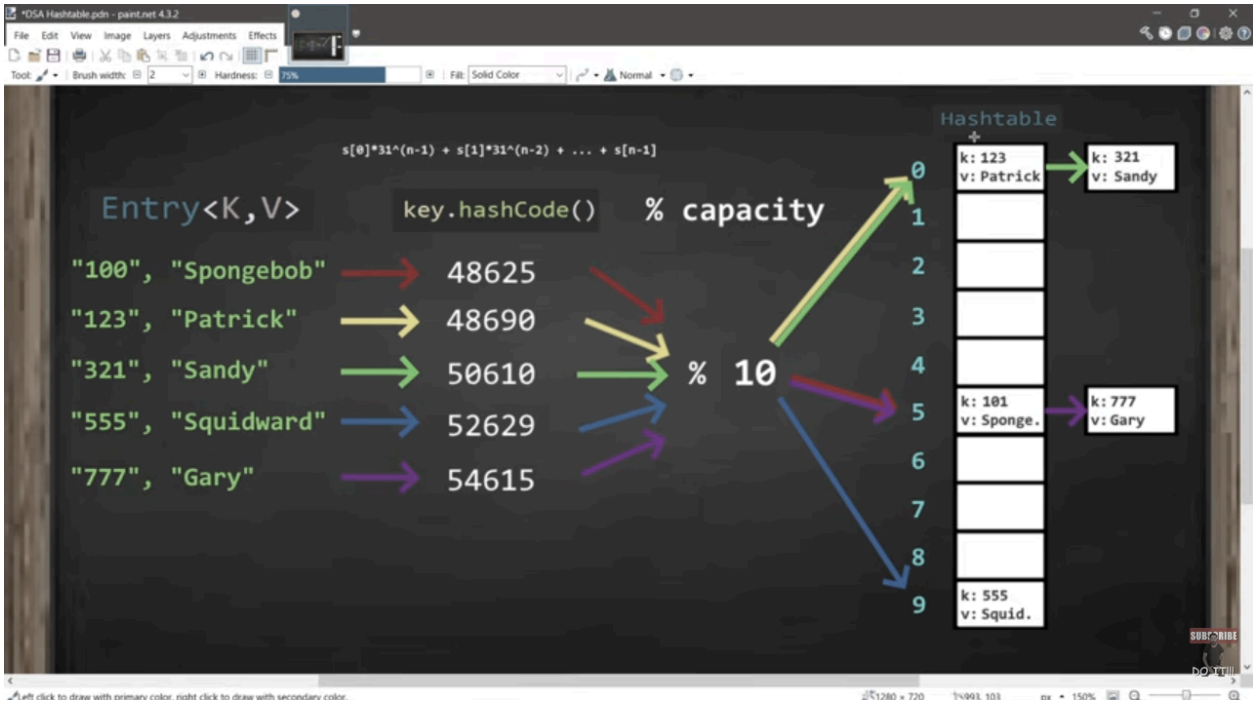
Time complexity of common operations

Operation	Complexity	Why
Access by index	O(1)	Direct address calculation
Search (unsorted)	O(n)	Must check elements one by one
Search (sorted, binary search)	O(log n)	Halve the search space each step
Insert/delete at the end	O(1) amortized	Just write to the next slot (dynamic arrays)
Insert/delete at the start or middle	O(n)	Every element after the gap must shift over

That last row is the main weakness: inserting at index 0 means physically sliding every other element one slot to the right, since there's no "gap" in contiguous memory.

Hash Tables





Operation / Aspect	Average case	Worst case
Access / lookup	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)
Search by value	O(n)	O(n)
Space complexity	**O(n)**	**O(n)**

https://www.youtube.com/watch?v=FsfRsGFHuv4
